

# D10 — Frontend de Pagamentos Pronto

Componentes UI independentes de gateway

Checkout, status, dashboard, conciliacao

Pronto para Stripe, Woovi, Pagarme, Mercado Pago ou qualquer outro

Projeto Aura — Plataforma de Gestao de Eventos

Documento 10 de 10 — Specdrive Tecnico



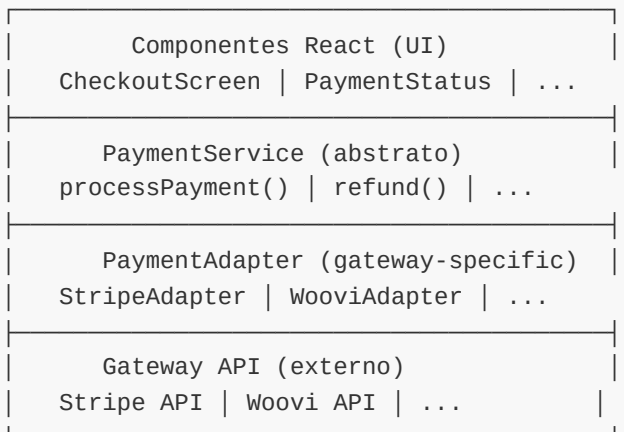
# 1. Visao Geral

---

Este documento define todos os componentes de frontend relacionados a pagamentos que podem ser desenvolvidos **agora**, sem depender da escolha do gateway. A arquitetura é abstraida: o componente de checkout captura os dados e chama um `processPayment()` generico, que sera implementado quando o gateway for definido.

**Principio Gateway-Agnostic:** Todos os componentes neste documento funcionam com qualquer gateway de pagamento. A integracao especifica fica em uma unica camada de adapter ( `src/api/paymentAdapter.ts` ), que sera implementada quando o gateway for escolhido.

## 1.1 Arquitetura de Abstracao



## 2. Componentes de Checkout

---

### 2.1 PaymentMethodSelector

`src/components/payments/PaymentMethodSelector.tsx`

### Props

- `availableMethods: PaymentMethod[]` - Lista de metodos habilitados
- `selectedMethod: string` - Metodo selecionado
- `onSelect: (method: string) => void` - Callback de selecao
- `disabled?: boolean` - Estado desabilitado

### Interface Visual

Grid de cartoes selecionaveis (2-3 colunas desktop, 1 coluna mobile). Cada cartao exibe:

- Icone do metodo (cartao de credito, Pix, boleto)
- Nome do metodo ("Cartao de Credito", "Pix", "Boleto")
- Subtitulo ("Ate 12x", "Aprovacao instantanea", "Compensacao em 1-2 dias")
- Bandeiras aceitas (Visa, Mastercard, etc.) - apenas para cartao
- Estado selecionado: borda verde + checkmark
- Estado desabilitado: opacidade reduzida, cursor bloqueado

```

export interface PaymentMethod {
  id: 'credit_card' | 'pix' | 'boleto' | 'debit_card'
  name: string
  description: string
  icon: string
  brands?: string[]
  enabled: boolean
  maxInstallments?: number
  instantApproval: boolean
}

const DEFAULT_METHODS: PaymentMethod[] = [
  {
    id: 'credit_card',
    name: 'Cartao de Credito',
    description: 'Ate 12x sem juros',
    icon: 'CreditCard',
    brands: ['visa', 'mastercard', 'amex', 'elo', 'hipercard'],
    enabled: true,
    maxInstallments: 12,
    instantApproval: true,
  },
  {
    id: 'pix',
    name: 'Pix',
    description: 'Aprovacao instantanea',
    icon: 'QrCode',
    enabled: true,
    instantApproval: true,
  },
  {
    id: 'boleto',
    name: 'Boleto Bancario',
    description: 'Compensacao em 1-2 dias uteis',
    icon: 'Barcode',
    enabled: true,
    instantApproval: false,
  },
]

```

## 2.2 CreditCardForm

```
src/components/payments/CreditCardForm.tsx
```

### Props

- `onCardDataChange: (data: CardData) => void` - Callback com dados do cartao
- `allowInstallments?: boolean` - Habilita parcelamento
- `maxInstallments?: number` - Maximo de parcelas
- `totalAmount: number` - Valor total para calcular parcelas

### Campos do Formulario

- **Numero do cartao:** Input com mascara (0000 0000 0000 0000), validacao Luhn, deteccao automatica de bandeira (Visa comeca com 4, MC com 5, etc.)
- **Nome no cartao:** Input texto, maiusculas automaticas
- **Validade (MM/AA):** Input com mascara, validacao de data futura
- **CVV:** Input numerico 3-4 digitos, icone de ajuda com tooltip "Codigo de seguranca"
- **Parcelas:** Select dropdown com options calculadas dinamicamente baseado no valor total (1x de R100,00, 2x de R 50,00, etc.)

```
export interface CardData {
  number: string
  holderName: string
  expiryMonth: string
  expiryYear: string
  cvv: string
  installments: number
  brand?: 'visa' | 'mastercard' | 'amex' | 'elo' | 'hipercard'
}

// Validacao independente de gateway
export function validateCardData(data: CardData): string[] {
  const errors: string[] = []
  if (!luhnCheck(data.number.replace(/\s/g, ''))) errors.push('Numero de cartao invalido')
  if (data.holderName.length < 3) errors.push('Nome no cartao incompleto')
  if (!/^(0[1-9]|1[0-2])\.\d{2}$/.test(`${data.expiryMonth}/${data.expiryYear}`)) errors.push('Validade invalida')
  if (data.cvv.length < 3) errors.push('CVV invalido')
  return errors
}
```

## 2.3 PixPayment

src/components/payments/PixPayment.tsx

### Props

- `qrCodeData: string` - Payload do QR Code (copia e cola)
- `qrCodeImageUrl?: string` - URL da imagem do QR Code
- `expiresAt: string` - Data/hora de expiracao
- `amount: number` - Valor do pagamento
- `onCopyCode: () => void` - Callback ao copiar codigo
- `status: 'pending' | 'paid' | 'expired'` - Estado do pagamento

### Interface Visual - 3 Estados

#### 1. Pendente:

- QR Code centralizado (imagem ou canvas gerado localmente)
- Botao "Copiar codigo Pix" grande, verde
- Countdown timer circular mostrando tempo restante (tipicamente 30 min)
- Instrucoes: "Abra seu app bancario, escaneie o QR Code ou cole o codigo"
- Lista de bancos com logo (Itau, Nubank, Bradesco, etc.) - informativo

#### 2. Pago:

- Icone de checkmark animado em verde
- "Pagamento confirmado!"
- Botao "Ver meus ingressos"

#### 3. Expirado:

- Icone de relógio em vermelho
- "QR Code expirado"
- Botao "Gerar novo QR Code"

## 2.4 BoletoPayment

```
src/components/payments/BoletoPayment.tsx
```

### Props

- `barcode: string` - Linha digitavel
- `boletoUrl?: string` - URL do PDF do boleto
- `expiresAt: string` - Data de vencimento
- `amount: number` - Valor
- `onCopyBarcode: () => void` - Callback

### Interface Visual

- Linha digitavel em fonte monoespacada, grande, com botao "Copiar" ao lado
- Botao "Visualizar boleto" (abre PDF em nova aba)
- Botao "Enviar por email"
- Aviso: "O ingresso sera liberado apos a compensacao do pagamento (1-2 dias uteis)"
- Data de vencimento destacada

## 3. Tela de Checkout Completa

---

### 3.1 CheckoutScreen

```
src/pages/checkout/CheckoutScreen.tsx
```

**Layout: 2 colunas (desktop), 1 coluna (mobile)**

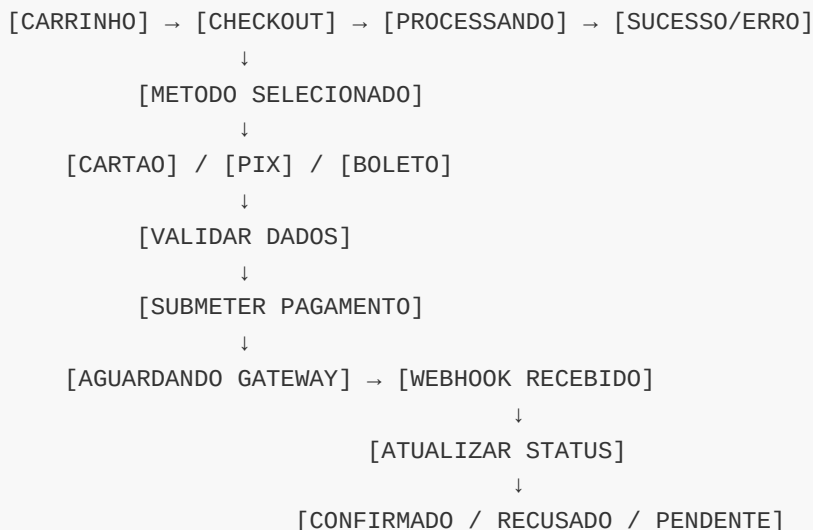
**Coluna esquerda (resumo do pedido):**

- Nome do evento + data/hora
- Lista de ingressos selecionados (tipo, quantidade, preco unitario, subtotal)
- Campo "Codigo de desconto" com botao "Aplicar"
- Linha de taxa de servico (transparente)
- **Total** em destaque

#### Coluna direita (pagamento):

- `PaymentMethodSelector`
- Formulario dinamico baseado no metodo selecionado (cartao, Pix, boleto)
- Dados do comprador (nome, email, CPF - preenchidos automaticamente se logado)
- Checkbox "Aceito os termos de uso" (link para modal)
- Botao "Pagar R\$ XXX,XX" grande, primario

## 3.2 Fluxo de Estados do Checkout



## 3.3 CheckoutSkeleton (Loading State)

```
src/components/payments/CheckoutSkeleton.tsx
```

Skeleton loader exibido enquanto os dados do evento e tipos de ingresso carregam. Deve ter a mesma estrutura visual da tela final para evitar layout shift:

- 2 colunas com shimmer effect
- Simula resumo do pedido e formulario de pagamento

## 4. Tela de Status de Pagamento

---

### 4.1 PaymentStatusScreen

```
src/pages/checkout/PaymentStatusScreen.tsx
```

#### Estados Visualis

##### 1. Processando:

- Spinner animado centralizado
- "Processando seu pagamento..."
- Subtexto: "Nao feche esta janela"

##### 2. Aprovado (Pix/Cartao):

- Animacao de confete ou checkmark
- "Pagamento confirmado!"
- Numero do pedido
- Email de confirmacao enviado para...
- Botao "Ver meus ingressos" (primario)
- Botao "Baixar ingressos" (secundario)
- Botao "Compartilhar no WhatsApp"

##### 3. Pendente (Boleto):

- Icone de relógio amarelo
- "Aguardando compensacao do boleto"
- Instrucoes de pagamento repetidas
- Botao "Ver boleto novamente"

**4. Recusado:**

- Ícone de X vermelho
- "Pagamento não autorizado"
- Possíveis razões (cartão recusado, saldo insuficiente, etc.)
- Botão "Tentar novamente"
- Botão "Escolher outro método de pagamento"

**4.2 RealtimeStatusUpdater**

```
src/components/payments/RealtimeStatusUpdater.tsx
```

Componente que escuta mudanças de status em tempo real via Supabase Realtime. Quando o webhook do gateway atualiza o status no banco, a tela atualiza automaticamente sem refresh:

```
// Hook generico - funciona com qualquer gateway
export function usePaymentStatus(orderId: string) {
  const queryClient = useQueryClient()

  useEffect(() => {
    const channel = supabase
      .channel(`order_${orderId}`)
      .on('postgres_changes', {
        event: 'UPDATE',
        schema: 'public',
        table: 'orders',
        filter: `id=eq.${orderId}`,
      }, (payload) => {
        queryClient.invalidateQueries({ queryKey: ['order', orderId] })
      })
      .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [orderId])
}
```

## 5. Dashboard Financeiro do Produtor

---

### 5.1 FinancialDashboard

```
src/pages/producer/FinancialDashboard.tsx
```

#### Secoes do Dashboard

##### Cards de KPI (topo):

- Total em vendas (mes atual) - com comparativo % vs mes anterior
- Total a receber (saldo pendente de repasse)
- Total recebido (mes atual)
- Quantidade de ingressos vendidos
- Taxa de conversao (visitas / vendas)

##### Graficos:

- Vendas por dia (line chart, ultimos 30 dias)
- Vendas por tipo de ingresso (pie/donut chart)
- Vendas por canal (organico, social, afiliado) - bar chart
- Comparativo mes a mes (bar chart)

##### Tabela de transacoes:

- Filtros: data, evento, status, metodo de pagamento
- Colunas: data, comprador, ingresso, metodo, valor, status, acoes
- Paginacao
- Botao "Exportar CSV/Excel"

### 5.2 WithdrawalRequest

```
src/components/payments/WithdrawalRequest.tsx
```

## Props

- `availableBalance: number` - Saldo disponivel para saque
- `pendingBalance: number` - Saldo pendente de compensacao
- `bankAccounts: BankAccount[]` - Contas bancarias cadastradas
- `onRequestWithdrawal: (amount: number, accountId: string) => void`

## Interface Visual

- Card com saldo disponivel em destaque (verde)
- Card com saldo pendente (amarelo, informativo)
- Input de valor do saque com validacao (max = availableBalance)
- Select de conta bancaria destino
- Botao "Solicitar saque" (desabilitado se valor > saldo)
- Historico de saques: data, valor, conta, status (pendente/processado)

```
export interface BankAccount {
  id: string
  bankCode: string
  bankName: string
  agency: string
  account: string
  type: 'checking' | 'savings'
  holderName: string
  holderDocument: string
  pixKey?: string
}

export interface WithdrawalRequest {
  id: string
  producerId: string
  amount: number
  bankAccountId: string
  status: 'pending' | 'processing' | 'completed' | 'failed'
  requestedAt: string
  processedAt?: string
}
```

## 6. Componentes de Administracao

---

### 6.1 GatewayConfigPanel

```
src/pages/admin/GatewayConfigPanel.tsx
```

#### Interface Visual

Painel administrativo para configurar o gateway de pagamento (quando for escolhido):

- Select: "Gateway de pagamento" (Stripe, Woovi, Pagar.me, Mercado Pago, Outro)
- Campos de API Key (mascarados, com botao "Revelar")
- Webhook URL (auto-gerada, com botao "Copiar")
- Configuracao de split: porcentagem da plataforma vs produtor
- Taxas por metodo de pagamento (cartao, Pix, boleto)
- Status de conexao: indicador verde/vermelho com "Testar conexao"
- Modo sandbox/producao toggle

### 6.2 TransactionMonitor

```
src/pages/admin/TransactionMonitor.tsx
```

#### Interface Visual - Visao Operacional

- Stream de transacoes em tempo real (cards descendo)
- Filtros: gateway, status, valor, periodo
- Métricas: total processado hoje, taxa de aprovacao, tempo medio de processamento
- Alertas: transacoes pendentes a mais de X minutos, chargebacks
- Detalhe da transacao: timeline completa (criado → processando → aprovado → repassado)

## 6.3 ReconciliationReport

src/pages/admin/ReconciliationReport.tsx

### Interface Visual - Conciliacao Financeira

- Seletor de periodo (mes/dia)
- Tabela: data, ID transacao, gateway, metodo, valor bruto, taxa gateway, taxa plataforma, valor liquido, status conciliacao
- Filtro de discrepancias: mostra apenas linhas onde "esperado != recebido"
- Botao "Exportar para contabilidade" (CSV com layout contabil)
- Resumo: total conciliado, total pendente, total divergente

## 7. Componentes de Reembolso

---

### 7.1 RefundModal

src/components/payments/RefundModal.tsx

#### Props

- `order: Order` - Pedido a ser reembolsado
- `isOpen: boolean`
- `onClose: () => void`
- `onConfirm: (refundData: RefundRequest) => void`
- `maxRefundable: number` - Valor maximo reembolsavel

#### Interface Visual

- Modal com overlay escuro
- Resumo do pedido: ingressos, comprador, data da compra
- Radio: "Reembolso total" ou "Reembolso parcial"
- Input de valor (aparece se parcial, com max = maxRefundable)
- Textarea: motivo do reembolso
- Aviso: "O reembolso sera processado em ate 5 dias uteis para cartao, instantaneo para Pix"
- Botoes: "Cancelar" (secundario), "Confirmar reembolso" (vermelho, com confirmacao dupla)

```
export interface RefundRequest {
  orderId: string
  amount: number          // 0 = total
  reason: string
  requestedBy: string     // user id
  type: 'full' | 'partial'
}

export interface RefundResult {
  refundId: string
  status: 'pending' | 'processing' | 'completed' | 'failed'
  estimatedDate?: string
  amountRefunded: number
}
```

### 7.2 RefundHistory

src/components/payments/RefundHistory.tsx

Tabela de historico de reembolsos por evento, com filtros e exportacao.

## 8. Hook Generico de Pagamento

---

## 8.1 usePayment (Gateway-Agnostic)

```
src/hooks/usePayment.ts
```

```

import { useMutation, useQuery } from '@tanstack/react-query'

export interface PaymentRequest {
  orderId: string
  method: 'credit_card' | 'pix' | 'boleto' | 'debit_card'
  cardData?: CardData
  customer: {
    name: string
    email: string
    document: string
  }
  amount: number
  metadata?: Record<string, any>
}

export interface PaymentResult {
  transactionId: string
  status: 'approved' | 'pending' | 'declined' | 'error'
  method: string
  qrCodeData?: string // Para Pix
  qrCodeImageUrl?: string
  boletoUrl?: string // Para boleto
  boletoBarcode?: string
  message?: string // Mensagem de erro ou info
  processedAt: string
}

// Hook generico - o adapter implementa a logica especifica
export function usePayment() {
  const processPayment = useMutation({
    mutationFn: async (request: PaymentRequest): Promise<PaymentResult> => {
      const { data, error } = await supabase.functions.invoke('process-payment', {
        body: request,
      })
      if (error) throw error
      return data
    },
  })

  const checkStatus = useQuery({
    queryKey: ['payment-status'],
    queryFn: async (transactionId: string) => {
      const { data } = await supabase.functions.invoke('payment-status', {
        body: { transactionId },
      })
      return data
    },
    enabled: false,
  })

  return {

```

**Como funciona:** O frontend chama `supabase.functions.invoke('process-payment')` generico. A Edge Function `process-payment` e quem decide qual adapter chamar baseado na configuracao do gateway. O frontend nao sabe nem precisa saber se e Stripe, Woovi ou outro.

## 9. Estrutura de Arquivos

```
src/
├── components/payments/
│   ├── PaymentMethodSelector.tsx    # Selecao de metodo (cartao/Pix/boleto)
│   ├── CreditCardForm.tsx           # Formulario de cartao + parcelas
│   ├── PixPayment.tsx               # QR Code + copia e cola + timer
│   ├── BoletoPayment.tsx            # Linha digitavel + PDF
│   ├── CheckoutSummary.tsx          # Resumo do pedido (coluna esquerda)
│   ├── CheckoutSkeleton.tsx         # Loading state do checkout
│   ├── PaymentStatus.tsx            # Status: processando/aprovado/recusado
│   ├── RealtimeStatusUpdater.tsx    # Atualizacao em tempo real
│   ├── RefundModal.tsx              # Modal de reembolso
│   ├── RefundHistory.tsx            # Historico de reembolsos
│   ├── InstallmentCalculator.tsx    # Calculo de parcelas
│   ├── SecureCardInput.tsx          # Input de cartao com validacao
│   └── index.ts                     # Exports
├── pages/checkout/
│   ├── CheckoutScreen.tsx           # Tela completa de checkout
│   ├── PaymentStatusScreen.tsx      # Tela de status pos-pagamento
│   └── OrderConfirmation.tsx         # Confirmacao com ingressos
├── pages/producer/
│   ├── FinancialDashboard.tsx        # Dashboard financeiro
│   ├── TransactionsList.tsx          # Lista de transacoes
│   └── WithdrawalRequest.tsx         # Solicitacao de saque
├── pages/admin/
│   ├── GatewayConfigPanel.tsx        # Configuracao do gateway
│   ├── TransactionMonitor.tsx        # Monitor de transacoes
│   └── ReconciliationReport.tsx      # Relatorio de conciliacao
├── hooks/
│   ├── usePayment.ts                # Hook generico de pagamento
│   ├── useInstallments.ts           # Calculo de parcelas
│   ├── usePixTimer.ts               # Countdown do QR Code
│   └── useCardValidation.ts          # Validacao de cartao (Luhn, etc)
├── types/
│   └── payments.ts                  # Todas as interfaces
└── api/
    └── paymentAdapter.ts             # Adapter (implementar quando gateway definido)
```

## 10. Checklist de Implementacao

---

**Tabela 1** Ordem de Implementacao dos Componentes de Pagamento

| Ordem | Componente                                                               | Esforco | Dependencia                                                    |
|-------|--------------------------------------------------------------------------|---------|----------------------------------------------------------------|
| 1     | <code>types/payments.ts</code> - Todas as interfaces                     | 2h      | Nenhuma                                                        |
| 2     | <code>useCardValidation.ts</code> - Validacao Luhn, bandeiras            | 3h      | Nenhuma                                                        |
| 3     | <code>useInstallments.ts</code> - Calculo de parcelas                    | 2h      | Nenhuma                                                        |
| 4     | <code>SecureCardInput.tsx</code> - Input com mascara                     | 4h      | <code>useCardValidation</code>                                 |
| 5     | <code>PaymentMethodSelector.tsx</code>                                   | 4h      | Nenhuma                                                        |
| 6     | <code>CreditCardForm.tsx</code>                                          | 6h      | <code>SecureCardInput</code> ,<br><code>useInstallments</code> |
| 7     | <code>PixPayment.tsx</code> - QR Code + timer                            | 5h      | <code>usePixTimer</code>                                       |
| 8     | <code>BoletoPayment.tsx</code>                                           | 3h      | Nenhuma                                                        |
| 9     | <code>CheckoutSummary.tsx</code> - Resumo do pedido                      | 4h      | Nenhuma                                                        |
| 10    | <code>CheckoutScreen.tsx</code> - Tela completa                          | 8h      | Todos acima                                                    |
| 11    | <code>CheckoutSkeleton.tsx</code> - Loading                              | 2h      | <code>CheckoutScreen</code>                                    |
| 12    | <code>PaymentStatus.tsx</code> +<br><code>PaymentStatusScreen.tsx</code> | 6h      | Nenhuma                                                        |
| 13    | <code>RealtimeStatusUpdater.tsx</code>                                   | 3h      | Supabase Realtime                                              |
| 14    | <code>FinancialDashboard.tsx</code>                                      | 10h     | Graficos (recharts)                                            |
| 15    | <code>TransactionsList.tsx</code>                                        | 6h      | Nenhuma                                                        |
| 16    | <code>WithdrawalRequest.tsx</code>                                       | 5h      | Nenhuma                                                        |

|              |                                     |                        |                   |
|--------------|-------------------------------------|------------------------|-------------------|
| 17           | RefundModal.tsx + RefundHistory.tsx | 6h                     | Nenhuma           |
| 18           | GatewayConfigPanel.tsx              | 4h                     | Nenhuma           |
| 19           | TransactionMonitor.tsx              | 6h                     | Supabase Realtime |
| 20           | ReconciliationReport.tsx            | 5h                     | Nenhuma           |
| <b>Total</b> | <b>20 componentes</b>               | <b>~94h (~12 dias)</b> |                   |

## 11. Decisões de Design que Facilitam a Integração Futura

### 11.1 Contratos de API Pre-definidos

Todos os componentes usam interfaces TypeScript que não mudam independente do gateway. Quando o gateway for escolhido, apenas o adapter precisa ser implementado:

```
// src/api/paymentAdapter.ts
// Implementar APOS escolher o gateway

export interface PaymentAdapter {
  processPayment(request: PaymentRequest): Promise<PaymentResult>
  checkStatus(transactionId: string): Promise<PaymentStatus>
  requestRefund(refund: RefundRequest): Promise<RefundResult>
  generatePix(orderId: string, amount: number): Promise<PixData>
  generateBoleto(orderId: string, amount: number, customer: Customer): Promise<BoletoData>
}

// Exemplo: StripeAdapter (futuro)
export class StripeAdapter implements PaymentAdapter { /* ... */ }

// Exemplo: WooviAdapter (futuro)
export class WooviAdapter implements PaymentAdapter { /* ... */ }
```

## 11.2 Feature Flags para Métodos de Pagamento

```
// src/config/paymentFeatures.ts
// Habilita/desabilita métodos sem mudar código

export const PAYMENT_FEATURES = {
  credit_card: true,
  pix: true,
  boleto: true,
  debit_card: false,      // Desabilitado até contratar gateway
  installments: true,
  max_installments: 12,
  refund_enabled: true,
  partial_refund: true,
  withdrawal_enabled: false, // Desabilitado até Fase 2
}

// Uso no componente:
// {PAYMENT_FEATURES.pix && <PixPayment ... />}
```

## 11.3 Estados de Pagamento Normalizados

O frontend trabalha com estados normalizados que mapeiam para qualquer gateway:

**Tabela 2** Mapeamento de Estados Normalizados

| Estado Aura (normalizado) | Stripe                       | Woovi (Pix)        | Woovi (Boleto)     |
|---------------------------|------------------------------|--------------------|--------------------|
| pending                   | requires_action / processing | pending            | pending            |
| approved                  | succeeded                    | completed          | completed          |
| declined                  | requires_payment_method      | cancelled          | cancelled          |
| refunded                  | refunded                     | refunded           | refunded           |
| partially_refunded        | partially_refunded           | partially_refunded | partially_refunded |
| expired                   | -                            | expired            | expired            |

## 12. Resumo do Specdrive Completo

Tabela 3 Specdrive Aura - Documentos Finais

| Doc | Titulo                       | Paginas | Conteudo                                     |
|-----|------------------------------|---------|----------------------------------------------|
| D1  | Inventario Frontend          | 19      | 130+ arquivos, 66 paginas, 82 componentes    |
| D2  | Gap Analysis                 | 12      | 30 tabelas, 10 Edge Functions, 9 integracoes |
| D3  | Analise Estrategica          | 8       | Decisoes: taxa 5-7%, SaaS, ARR R\$8.88M      |
| D4  | Frontend Melhorias           | 9       | UX, performance, a11y, SEO, novas features   |
| D5  | Supabase Spec                | 27      | SQL schema, RLS, triggers, Edge Functions    |
| D6  | Arquitetura de Sistemas      | 8       | Supabase, Vercel, Hostinger, gateways        |
| D7  | Organizacao de Agentes       | 10      | 3 agentes, comandos, skills                  |
| D8  | Implementacao e Deploy       | 31      | Codigo real, configs, checklist              |
| D9  | Analise Estrategica e Riscos | 16      | Riscos criticos, mitigacoes, plano de acao   |
| D10 | Frontend de Pagamentos       | 18      | 20 componentes gateway-agnostic              |

**Total do Specdrive: 10 documentos, 158+ paginas tecnicas.** O D10 entrega 20 componentes de pagamento prontos para implementacao, totalmente independentes da escolha do gateway. Quando o gateway for definido, apenas o adapter ( `src/api/paymentAdapter.ts` ) e as Edge Functions precisam ser implementados — todo o UI ja estara pronto.